

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования



НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ

УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий

Кафедра «Вычислительные системы и технологии»

ОТЧЕТ

по лабораторной работе №2

по дисциплине

Языки интернет-программирования

РУКОВОДИТЕЛЬ:

(подпись)

Яшнов М.Д.
(фамилия, и.,о.)

СТУДЕНТ:

(подпись)

Ивашина Е.Р.
(фамилия, и.,о.)

23-ИИ
(шифр группы)

Работа защищена «__» _____

С оценкой _____

Нижний Новгород
2026

Ссылка на GitHub: <https://github.com/luft15/Books-club>

Тема лабораторной работы

Реализация проекта с серверным рендерингом страниц, работа с базой данных и безопасностью.

Цель

- Освоить серверный рендеринг HTML-страниц
- Научиться безопасно работать с базой данных через чистый SQL
- Использовать архитектуру MVC и принципы DI
- Освоить контейнеризацию с Docker для приложения и БД
- Реализовать CRUD операции и взаимосвязь таблиц

1. - Все HTML-страницы должны формироваться на сервере. Клиентский JavaScript разрешён только для интерактивности

Из-за того, что основным языком программирования выбран Python, то будет использоваться фреймворк Flask

Src/controllers/index_controller.py:

```
Books-club > src > controllers > index_controller.py > ...
1  from flask import render_template
2  from app import app
3
4
5  @app.route("/")
6  def main_page():
7      return render_template("index.html")
8
9
```

Src/controllers/info_controller.py:

```
1  from flask import render_template
2  from app import app
3
4  @app.route("/info")
5  def information():
6      return render_template("information.html")
7
```

Src/controllers/plan_controller.py:

```
looks-club > src > controllers > plan_controller.py > plan
1  from flask import render_template
2  from app import app
3
4
5  @app.route("/plan")
6  def plan():
7      return render_template("plan.html")
8
```

Каждый контроллер подключается в src/app.py:

```
1  from flask import Flask
2
3  # from db.db import connect_db
4
5
6  app = Flask(__name__)
7
8  # db_con = connect_db()
9
10 from controllers.index_controller import *
11 from controllers.info_controller import *
12 from controllers.plan_controller import *
13
14 if __name__ == "__main__":
15     app.run(
16         host="0.0.0.0",
17         port=80,
18         debug=True
19     )
```

2. - Развертывание БД и приложения через Docker Compose

Созданы docker_compose:

```

Books-club > 🐳 docker-compose.yml
1  services:
2    db_booksclub:
3      image: postgres:17-alpine
4      environment:
5        POSTGRES_USER: ${DB_USER}
6        POSTGRES_DB: ${DB_NAME}
7        POSTGRES_PASSWORD: ${DB_PASSWORD}
8      ports:
9        - "${DB_PORT}:5432"
10     volumes:
11       - pgdata:/var/lib/postgresql/data
12       - ./src/database:/docker-entrypoint-initdb.d
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
15       interval: 10s
16       timeout: 5s
17       retries: 5
18     container_name: db_booksclub
19
20     backend_booksclub:
21       build:
22         context: .
23         dockerfile: Dockerfile
24       environment:
25         DB_HOST: db_booksclub
26         DB_PORT: ${DB_PORT}
27         DB_NAME: ${DB_NAME}
28         DB_USER: ${DB_USER}
29         DB_PASSWORD: ${DB_PASSWORD}
30         FLASK_ENV: development
31         FLASKDEBUG: 1
32       ports:
33         - "8080:80"
34       volumes:
35         - ./src:/app
36         - /app/__pycache__
37       depends_on:
38         db_booksclub:
39           condition: service_healthy
40       restart: on-failure
41       working_dir: /app
42       container_name: backend_flask_booksclub
43
44     volumes:
45       pgdata:

```

И создан Dockerfile:

```
Books-club > Dockerfile
1 FROM python:3.10-alpine
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY src/ .
9 EXPOSE 80
10
11 CMD ["python", "app.py"]
```

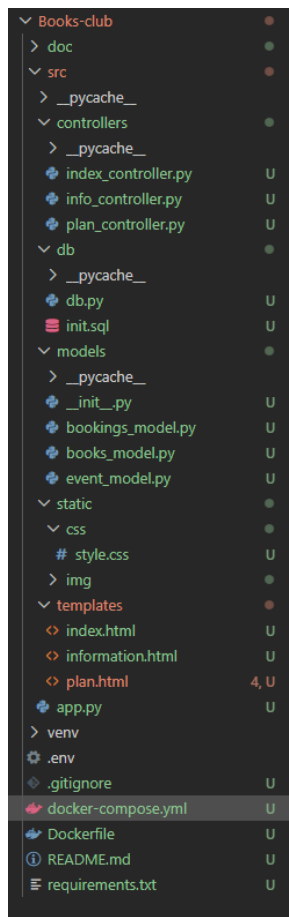
Также для нормальной работы докера нужен был файл requirements.txt. Он был создан командой `pip freeze > requirements.txt`

3. - Использование MVC архитектуры, контроллеры обрабатывают запросы, модели — работу с БД, представления — шаблоны HTML

Эта архитектура нужна чтобы отделять логику. Модели работают с базой данных — контроллеры обрабатывают запросы, валидируют их.

Представления — всё так же работают с HTML, но теперь в них есть Jinja

Архитектура:



Контроллер(главный!):

```
@app.route("/plan", methods=['GET', 'POST'])
def plan():
    if request.method == "POST":
        name = request.form.get('user_name', "").strip()
        phone = request.form.get('user_phone', "").strip()
        email = request.form.get('user_email', "").strip()
        date_str = request.form.get('date', '').strip()
        time_slot = request.form.get('time_slot', "").strip()
        comment = request.form.get('comment', "").strip()

        if len(name) < 3:
            return jsonify({"status": "error", "errors": "имя должно содер
        if not re.match(r'^^@]+@^@\+\.[^@]+\$', email):
            return jsonify({"status": "error", "errors": "Неверный email"}
        if not date_str:
            return jsonify({"status": "error", "errors": "Выберите дату во

        event = EventModel.get_by_date(date_str)
        if not event:
            return jsonify({"status": "error", "errors": "Событие на эту д

        BookingModel.create(
            event_id=event['id'],
            user_name=name,
            user_phone=phone,
            user_email=email,
            time_slot=time_slot,
            comment=comment
        )

        print(f'- запись сохранена: {name} на {date_str} {time_slot}')
        return jsonify({"status": "ok"})
```

Модели (все написаны на чистом SQL):

```

class EventModel:
    @staticmethod
    def get_all():
        query = """
        SELECT
            e.id,
            e.book_id,
            e.event_date,
            e.description,
            e.max_participants,
            e.created_at,
            b.title AS book_title,
            b.author AS book_author,
            b.year AS book_year
        FROM events e
        JOIN books b ON e.book_id = b.id
        ORDER BY e.event_date;
        """
        return execute_query(query, fetch=True)

    @staticmethod
    def get_by_id(event_id):
        query = """
        SELECT
            e.id,
            e.book_id,
            e.event_date,
            e.description,
            e.max_participants,
            e.created_at,
            b.title AS book_title,
            b.author AS book_author,
            b.year AS book_year
        FROM events e
        JOIN books b ON e.book_id = b.id
        WHERE e.id = %s;
        """
        result = execute_query(query, (event_id,), fetch=True)
        return result[0] if result else None

    @staticmethod
    def create(book_id, event_date, description, max_participants=20):
        query = """
        INSERT INTO events (

```

Представления:

```

138         <a href="#" class="social-link">Te
139         <a href="#" class="social-link">VK
140         <a href="#" class="social-link">Yo
141     </div>
142 </div>
143 </div>
144 <div class="footer-bottom">
145     <p>&copy; 2025 BOOKSclub. Все права защище
146 </div>
147 </div>
148 </footer>
149
150 <script>
151     // События, переданные из Flask
152     const eventsByDate = {{{ services|tojson }}};
153
154     function buildMonthCalendar(year, month, eventsMap
155     const container = document.getElementById(cont
156     if (!container) return;

```

4. - CRUD операции минимум для одной сущности

```
Books-club > src > models > event_model.py > EventModel > get_by_id
4  class EventModel:
5      def get_by_id(event_id):
42      return result[0] if result else None
43
44      @staticmethod
45      def create(book_id, event_date, description, max_participants=20):
46          query = """
47              INSERT INTO events (
48                  book_id,
49                  event_date,
50                  description,
51                  max_participants
52              )
53              VALUES (%s, %s, %s, %s);
54          """
55          execute_query(
56              query,
57              (book_id, event_date, description, max_participants)
58          )
59
60      @staticmethod
61      def update(event_id, book_id, event_date, description, max_participants):
62          query = """
63              UPDATE events
64              SET
65                  book_id = %s,
66                  event_date = %s,
67                  description = %s,
68                  max_participants = %s
69              WHERE id = %s;
70          """
71          execute_query(
72              query,
73              (
74                  book_id,
75                  event_date,
76                  description,
77                  max_participants,
78                  event_id
79              )
80          )
81
```



```

@staticmethod
def delete(event_id):
    query = "DELETE FROM events WHERE id = %s;"
    execute_query(query, (event_id,))

@staticmethod
def get_available_slots(event_id):
    query = """
        SELECT
            e.max_participants
            - COUNT(bk.id) AS available_slots
        FROM events e
        LEFT JOIN bookings bk ON e.id = bk.event_id
        WHERE e.id = %s
        GROUP BY e.id, e.max_participants;
    """
    result = execute_query(query, (event_id,), fetch=True)
    return result[0]["available_slots"] if result else 0

@staticmethod
def get_by_date(date_str):
    query = """
        SELECT e.id, e.book_id, e.event_date, e.description, e.max_participants,
            b.title AS book_title, b.author AS book_author
        FROM events e
        JOIN books b ON e.book_id = b.id
        WHERE e.event_date = %s
        LIMIT 1;
    """
    result = execute_query(query, (date_str,), fetch=True)
    return result[0] if result else None

```

5. - Минимум три таблицы по предметной области. Не менее одной связи между таблицами (один-ко-многим или многие-ко-многим). Если нет идеи для предметной области — сделать таблицы для статистики действий/логов/переходов. Все таблицы должны быть в нормальных формах

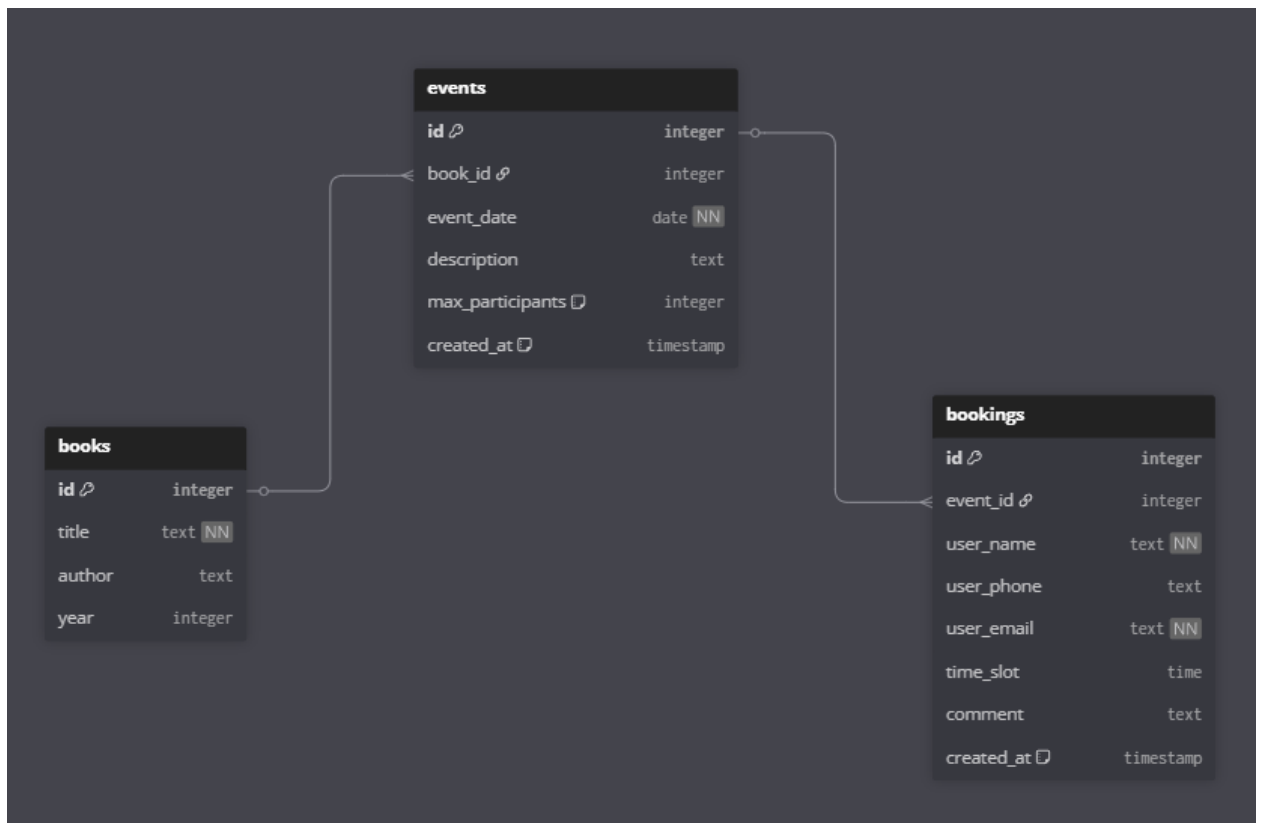
Был создан init.sql с таблицами и данными:

```

books-club > src > db > init.sql
1  -- 1. Создание таблиц
2  CREATE TABLE books (
3      id SERIAL PRIMARY KEY,
4      title TEXT NOT NULL,
5      author TEXT,
6      year INTEGER
7  );
8
9  CREATE TABLE events (
10     id SERIAL PRIMARY KEY,
11     book_id INTEGER REFERENCES books(id) ON DELETE CASCADE,
12     event_date DATE NOT NULL,
13     description TEXT,
14     max_participants INTEGER DEFAULT 20,
15     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
16 );
17
18 CREATE TABLE bookings (
19     id SERIAL PRIMARY KEY,
20     event_id INTEGER REFERENCES events(id) ON DELETE CASCADE,
21     user_name TEXT NOT NULL,
22     user_phone TEXT,
23     user_email TEXT NOT NULL,
24     time_slot TIME,
25     comment TEXT,
26     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
27 );
28
29 -- 2. Добавляем тестовые книги
30 INSERT INTO books (title, author, year) VALUES
31 ('Джейн Эйр', 'Шарлотта Бронте', 1847),
32 ('Бесы', 'Ф.М. Достоевский', 1872),
33 ('Лигея', 'Эдгар Аллан По', 1838);
34
35 -- 3. Добавляем тестовые события
36 INSERT INTO events (book_id, event_date, description, max_participants) VALUES
37 (1, '2026-03-20', 'Zoom-встреча, обсуждение «Джейн Эйр»', 30),
38 (2, '2026-02-15', 'Офлайн в клубе на ул. Книжная, 10', 20),
39 (3, '2026-03-06', 'Поэтический вечер по рассказу «Лигея»', 15);
40
41 -- 4. Добавляем тестовые записи пользователей
42 INSERT INTO bookings (event_id, user_name, user_phone, user_email, time_slot, co

```

Схема:

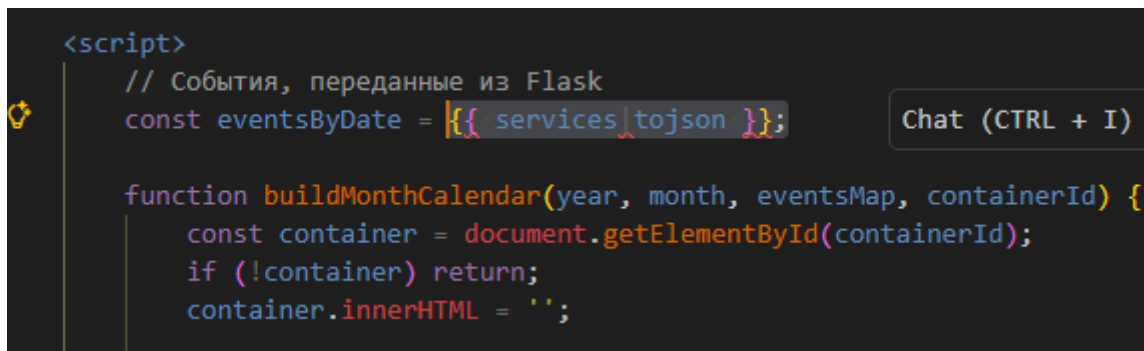


6. - Защита от SQL-инъекций обязательна (использование параметризованных запросов)

Она используется! Пример на скриншоте:

```
@staticmethod
def create(event_id, user_name, user_email, user_phone=None, time_slot=None, comment=None):
    query = """
        INSERT INTO bookings (
            event_id, user_name, user_phone, user_email, time_slot, comment
        ) VALUES (%s, %s, %s, %s, %s, %s);
    """
    execute_query(query, (event_id, user_name, user_phone, user_email, time_slot, comment))
```

7. - Обязательная защита от XSS



```
<script>
// События, переданные из Flask
const eventsByDate = {{ services|tojson }};

function buildMonthCalendar(year, month, eventsMap, containerId) {
  const container = document.getElementById(containerId);
  if (!container) return;
  container.innerHTML = '';
}
```

Фильтр `tojson` экранирует все специальные символы (включая `<`, `>`, `&`, `'`, `"`), превращая данные в безопасный JSON-совместимый JavaScript-код.

В любом месте, где будет написан `{{ переменная }}` – внутри HTML JINJA автоматически экранирует символы. Так можно предотвратить атаку на подобию `"><script>alert('XSS')</script>`

8. - Проверка и фильтрация входных данных

Проверка и фильтрация производится внутри контроллера, согласно архитектуре MVC. Чтобы модель работала только с базой данных, а контроллер фильтровал входные данные. В перспективе – можно вынести в отдельную функцию для лучшей читаемости кода.



```
@app.route("/plan", methods=['GET', 'POST'])
def plan():
    if request.method == "POST":
        name = request.form.get('user_name', "").strip()
        phone = request.form.get('user_phone', "").strip()
        email = request.form.get('user_email', "").strip()
        date_str = request.form.get('date', '').strip()
        time_slot = request.form.get('time_slot', "").strip()
        comment = request.form.get('comment', "").strip()

        if len(name) < 3:
            return jsonify({"status": "error", "errors": "имя должно быть не менее 3 символов"})
        if not re.match(r'^[^\@]+\@[^\@]+\.[^\@]+$', email):
            return jsonify({"status": "error", "errors": "Неверный email"})
        if not date_str:
            return jsonify({"status": "error", "errors": "Выберите дату"})

        event = EventModel.get_by_date(date_str)
        if not event:
            return jsonify({"status": "error", "errors": "Событие не найдено"})

        BookingModel.create(
            event_id=event['id'],
            user_id=request.user.id,
            phone=phone,
            time_slot=time_slot,
            comment=comment
        )
        return jsonify({"status": "success", "message": "Бронирование успешно создано"})
```

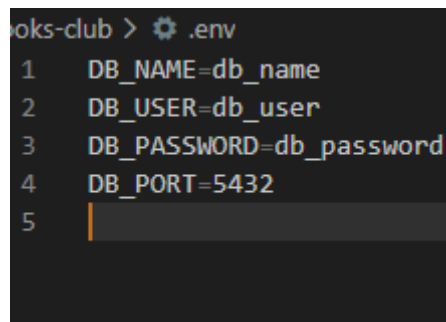
Так, например, `.strip()` убирает лишние пробелы, чтобы предотвратить нежеланный ввод.

9. - Dependency Injection. Использовать DI для подключения к БД, логгера, сервисов и т.д.

Задание увидела слишком поздно, у меня уже были реализованы статистические методы.

Отсутствие DI допустимо, потому что в проекте отсутствует сложная бизнес-логика. Использованный статический подход модели и **прямой вызов `execute_query()`** достаточен для демонстрации работы с БД и веб-форм. Однако в следующей лабораторной работе, где нужно будет обновить модели и переписать всё на ORM, появится DI

10.- Все параметры (строки подключения к БД, ключи) хранятся в переменных окружения или конфигурационных файлах



```
oks-club > ⚙ .env
1 DB_NAME=db_name
2 DB_USER=db_user
3 DB_PASSWORD=db_password
4 DB_PORT=5432
5
```

Также этот файл был дообавлен в `.gitignore`, чтобы случайно не запустить его.